HypArr

Computations with hyperplane arrangements

0.1

24 March 2026

Paul Mücksch

Paul Mücksch
Email: paul.muecksch+uni@gmail.com
Homepage: https://paulmuecksch.github.io
Address: Welfengarten 1, 30167 Hannover, Germany

Contents

1 Introduction **3**

2 Hyperplane arrangements **4**
2.1 Attributes of arrangements 4
2.2 Properties of arrangements 6
2.3 Attributes of geometric lattices 6
2.4 Properties of geometric lattices 7
2.5 Operations 7
2.6 Global methods 8
2.7 Constructors 10

3 Free arrangements **12**
3.1 Freeness properties 12

4 Milnor fibers **13**
4.1 Complexes 13

5 Further arrangement properties **15**
5.1 Formal arrangements 15

6 Oriented Matroids **17**
6.1 Attributes 17
6.2 Properties 21
6.3 Operations 21
6.4 Global methods 22
6.5 Constructors 22

7 Special arrangements **24**
7.1 Global methods 24

8 Chamber and tope posets **26**
8.1 Operation 26
8.2 Attributes 26

References **27**

Index **28**

Chapter 1

Introduction

HYPARR is a GAP 4 package for computations with hyperplane arrangements (see [OT92]), oriented matroids (see [BLVS+99]), their combinatorial and topological invariants, in particular geometric lattices, face posets, Milnor fibers and complements.

Chapter 2

Hyperplane arrangements

Let $\mathbb{K}$ be a field. A hyperplane arrangment is a finite set $\mathscr{A} = \{H_1, \ldots, H_n\}$ of hyperplanes in a finite dimensional $\mathbb{K}$-vector space V.

2.1 Attributes of arrangements

2.1.1 Dimension (for IsHyperplaneArrangement)

▷ `Dimension(A)` (attribute)

Returns: a non-negative integer.

Returns the dimension of the ambient vector space in which the hyperplane arrangement `A` is defined.

2.1.2 Roots (for IsHyperplaneArrangement)

▷ `Roots(A)` (attribute)

Returns: A list of vectors defining the hyperplanes of `A`.

Returns the list of defining linear forms (also called *roots*) of the hyperplanes in the arrangement $\mathscr{A}$. Each vector represents a hyperplane by the linear equation

$$r_1x_1 + \cdots + r_nx_n = 0.$$

2.1.3 IntersectionLattice (for IsHyperplaneArrangement)

▷ `IntersectionLattice(A)` (attribute)

Returns: An object of type `IsGeomLattice`.

Computes the intersection lattice of the hyperplane arrangement $\mathscr{A}$.

The ground set of the lattice is represented level-by-level. The entry $L[k]$ contains all intersections of rank k.

Each lattice element is stored as a list of indices referring to hyperplanes in `Roots(A)`.

For example, the list `[1,3]` represents the intersection of the first and third hyperplane of the arrangement.

The algorithm incrementally constructs the lattice by adding hyperplanes one by one.

Example
```
gap> A := HyperplaneArrangement([[1,0],[0,1],[1,1]]);
gap> IntersectionLattice(A);
<Geometric lattice: 3 atoms, rank 2>
```

2.1.4 CharPoly (for IsHyperplaneArrangement)

▷ `CharPoly(A)` (attribute)

Returns: a univariate polynomial with integral coefficients.

Returns $\chi_{\mathscr{A}}$ the *characterisitc polynomial* of the arrangement $\mathscr{A}$.

Example
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> CharPoly(A);
t^3-6*t^2+11*t-6
```

2.1.5 ExpArr (for IsHyperplaneArrangement)

▷ `ExpArr(A)` (attribute)

Returns: a list of integers or fail

Returns *exponents* of the arrangement $\mathscr{A}$ if the characteristic polynomial factors over the integers.

Example
```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> ExpArr(A);
[ 1, 2 ]
```

2.1.6 MSetInvL (for IsHyperplaneArrangement)

▷ `MSetInvL(A)` (attribute)

Returns: An object of type IsGeomLattice.

Computes multiset invariants of the intersection lattice of the hyperplane arrangement `A`.

For each level of the lattice, the function records how many intersections occur with a given number of defining hyperplanes.

These invariants can be used to compare arrangements or detect combinatorial similarities between intersection lattices.

Example
```
gap> A:=HyperplaneArrangement([[1,0,0,],[0,1,0],[0,0,1],[1,1,1]]);
<HyperplaneArrangement: 4 hyperplanes in 3-space>
gap> MSetInvL(A);
[ [ [ 1, 4 ] ], [ [ 2, 6 ] ], [ [ 4, 1 ] ] ]
```

2.1.7 SalvettiComplex (for IsHyperplaneArrangement)

▷ `SalvettiComplex(A)` (attribute)

Returns: FacePoset

Constructs the Salvetti Complex [Sal87]. If `IsReal(A)` is false, then it returns `fail`.

Example
```
gap> A:=HyperplaneArrangement([[1,0],[0,1],[1,1]]);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
gap> SalvettiComplex(A);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

2.2 Properties of arrangements

2.2.1 IsReal (for IsHyperplaneArrangement)

▷ `IsReal(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is defined over the reals, i.e., if the entries of the defining linear forms are real.

2.2.2 HArrIsIrreducible (for IsHyperplaneArrangement)

▷ `HArrIsIrreducible(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is irreducible.

2.3 Attributes of geometric lattices

2.3.1 GLGroundSet (for IsGeomLattice)

▷ `GLGroundSet(L)` (attribute)

Returns: list of lists

Returns the ground set of the geometric lattice `L`.

2.3.2 GLAtoms (for IsGeomLattice)

▷ `GLAtoms(L)` (attribute)

Returns: list

Returns the set of atoms of `L`.

2.3.3 GLRank (for IsGeomLattice)

▷ `GLRank(L)` (attribute)

Returns: a non-negative integer

Returns the rank of `L`.

2.3.4 GLkFlats (for IsGeomLattice)

▷ `GLkFlats(L)` (attribute)

Returns: a function

Returns a function extracting the flats of rank k in `L`.

2.3.5 GLGraph (for IsGeomLattice)

▷ `GLGraph(L)` (attribute)

Returns: a graph

Computes the directed graph of the Hasse diagram of L.

2.3.6 GLAutGroup (for IsGeomLattice)

▷ `GLAutGroup(L)` (attribute)

Returns: a group

Computes the autmorphism group of L as a subgroup of `Sym(GLAtoms(L))`.

2.3.7 GLMoebius (for IsGeomLattice)

▷ `GLMoebius(L)` (attribute)

Returns: function

Returns the Moebius function of `L`.

2.3.8 GLCharPoly (for IsGeomLattice)

▷ `GLCharPoly(L)` (attribute)

Returns: function

Computes the characteristic polynomial of `L`.

2.4 Properties of geometric lattices

2.4.1 GLIsIrreducible (for IsGeomLattice)

▷ `GLIsIrreducible(L)` (property)

Returns: true or false.

Determines, if L is irreducible.

2.5 Operations

2.5.1 IsLEquiv (for IsHyperplaneArrangement,IsHyperplaneArrangement)

▷ `IsLEquiv(A, B)` (operation)

Returns: true or false

Determines if the arrangements `A` and `B` have isomorphic intersection lattices.

Example

```
gap> A:=Arr([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,3]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> B:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> IsLEquiv(A,B);
false
gap> A:=Arr([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[0,1,1],[1,1,1]]);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
```

```
gap> IsLEquiv(A,B);
true
```

2.5.2 Essentialization (for IsHyperplaneArrangement)

▷ `Essentialization(A)` (operation)

Returns: A hyperplane arrangement.

Computes the *essentialization* of the hyperplane arrangement `A`. An arrangement is called essential if the intersection of all its hyperplanes is the origin. If this is not the case, the function restricts the arrangement to a complementary subspace so that the resulting arrangement becomes essential.

Example

```
gap> A:=Arr([[1,0,0],[0,1,0],[1,1,0]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> B:=Essentialization(A); Roots(B);
<HyperplaneArrangement: 3 hyperplanes in 2-space>
[ [ 1, 0 ], [ 0, 1 ], [ 1, 1 ] ]
```

2.6 Global methods

2.6.1 DrawLatex3Arr

▷ `DrawLatex3Arr(A[, ps[, ip[, Hind[, disthv[, MarkHs]]]]])` (function)

Returns: A string.

Generates tikz-code for a nice projective picture of the real 3-arrangement. If x, y, z are the coordinates, by default, this is the 2-dim affine arrangement obtained by intersecting `A` with the plane $z = 1$.

The optional parameters are:

- `ps` a scaling factor,
- `ip` intersection points are drawn,
- `Hind` labels for the hyperplanes are added,
- `disthv` a vector giving the normal of the plane whith which to intersect `A`,
- `MarkHs` a list of indices of hyperplanes of `A`, these are drawn in another color.

The example below will look as follows (only in pdf).

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(DrawLatex3Arr(A));
\begin{tikzpicture}[scale=1.0]
\draw (-2.827,2.827) -- (2.827,-2.827);  % H_1
\draw (2.827,2.827) -- (-2.827,-2.827);  % H_2
\draw (-1.,3.873) -- (-1.,-3.873);  % H_3
\draw (1.,3.873) -- (1.,-3.873);  % H_4
\draw (3.873,-1.) -- (-3.873,-1.);  % H_5
\draw (3.873,1.) -- (-3.873,1.);  % H_6
```

```
\end{tikzpicture}
gap> Print(DrawLatex3Arr(A,1/2,true,true,[1,1,1],[1,2]));
\begin{tikzpicture}[scale=1.0]
\draw[color=red] (-3.559,1.827) -- (1.827,-3.559);  % H_1
\node at (1.975,-3.707) {\small $1$};
\draw[color=red] (2.827,2.827) -- (-2.827,-2.827);  % H_2
\node at (-2.970,-2.970) {\small $2$};
\draw (3.361,2.169) -- (-3.995,0.197);  % H_3
\node at (-4.198,0.143) {\small $3$};
\draw (-1.034,3.863) -- (1.034,-3.863);  % H_4
\node at (1.087,-4.057) {\small $4$};
\draw (2.169,3.361) -- (0.197,-3.995);  % H_5
\node at (0.143,-4.198) {\small $5$};
\draw (-3.863,1.034) -- (3.863,-1.034);  % H_6
\node at (4.057,-1.087) {\small $6$};

\fill[red] (-0.865,-0.865) circle[radius=2pt];  % P[ 1, 2 ]
\fill[red] (-2.366,0.634) circle[radius=2pt];  % P[ 1, 3, 6 ]
\fill[red] (1.732,1.732) circle[radius=2pt];  % P[ 2, 3, 5 ]
\fill[red] (0.634,-2.366) circle[radius=2pt];  % P[ 1, 4, 5 ]
\fill[red] (0.0,0.0) circle[radius=2pt];  % P[ 2, 4, 6 ]
\fill[red] (-0.317,1.183) circle[radius=2pt];  % P[ 3, 4 ]
\fill[red] (1.183,-0.317) circle[radius=2pt];  % P[ 5, 6 ]

\end{tikzpicture}
```

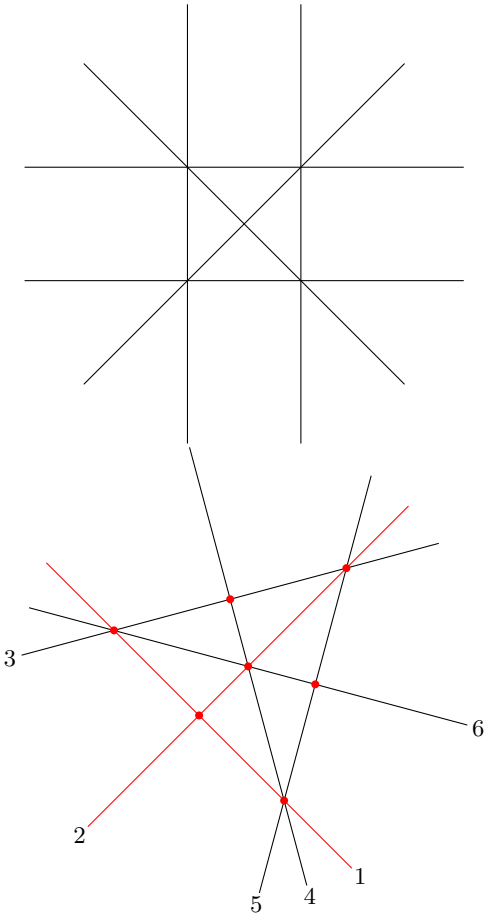

Figure 1: Example of pictures produced by `DrawLatex3Arr`.

2.6.2 HArrResHind

▷ `HArrResHind(A, i)` (function)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the `i`-th hyperplane of `Roots(A)`.

2.6.3 HArrResX

▷ `HArrResX(A, S)` (function)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the subspace spanned by the vectors in `S`.

2.7 Constructors

2.7.1 HyperplaneArrangement (for IsList)

▷ `HyperplaneArrangement(r)` (operation)

Returns: A hyperplane arrangement

Constructs a hyperplane arrangement from a list `R` of vectors representing defining linear forms of hyperplanes.

Each vector $r = [r_1, \ldots, r_n]$ corresponds to the hyperplane

$$r_1 x_1 + \cdots + r_n x_n = 0.$$

The vectors must lie in the same vector space.

Linearly dependent defining forms are removed automatically, so the resulting arrangement only stores pairwise linearly independent hyperplanes.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ] ]
```

Chapter 3

Free arrangements

The following describes functions to analyse freeness properties of arrangements such as *inductive freeness* due to H. Terao [Ter80] and *divisionell freeness* due to T. Abe [Abe16].

3.1 Freeness properties

3.1.1 IsInductivelyFree (for IsHyperplaneArrangement)

▷ `IsInductivelyFree(A)` (property)

Returns: true or false.

Determines whether the arrangement `A` is *inductively free*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsInductivelyFree(A);
true
```

3.1.2 IsDivisionallyFree (for IsHyperplaneArrangement)

▷ `IsDivisionallyFree(A)` (property)

Returns: true or false.

Determines whether the arrangement `A` is *divisionally free*.

Example

```
gap> A:=HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1],[1,1,0],[1,1,1]]);
<HyperplaneArrangement: 5 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
true
gap> A:=AGpql(3,3,3);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> IsDivisionallyFree(A);
false
```

Chapter 4

Milnor fibers

The Milnor fiber is the typical fiber $f^{-1}(1)$ of the Milnor fibration

$$f : \mathbb{C}^\ell \setminus \cup_{H\in\mathscr{A}} H \to \mathbb{C}^\times, z \mapsto \prod_{H\in\mathscr{A}} \alpha_H(z)$$

of the hyperplane arrangement $\mathscr{A}$, where α_H are defining linear forms.

The following describes functions to construct a regular cell complex having the homotopy type of the Milnor fiber from [MY25] and thus enables computation of homotopy invaraints using e.g. the HAP package [Ell26].

4.1 Complexes

4.1.1 MilnorFiberComplex (for IsOrientedMatroid)

▷ `MilnorFiberComplex(OM)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex of the combinatorial Milnor fiber of the oriented matroid *OM*.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> MCpx:=MilnorFiberComplex(O);
<FacePoset of dimension 1 with f-vector [ 3, 6 ]>
```

4.1.2 MilnorFiberComplex (for IsHyperplaneArrangement)

▷ `MilnorFiberComplex(A)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex having the homotopy type of Milnor fiber of the arrangement *A*.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
```

4.1.3 FPtoCWCpx

▷ `FPtoCWCpx(FP)` (function)

Returns: HAP CW Complex

Converts a HYPARR `FacePoset` into a HAP `RegularCWComplex` for further computations of topological invariants.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
gap> MCW := FPtoCWCpx(MCpx);
Regular CW-complex of dimension 2
gap> Homology(MCW,0);
[ 0 ]
gap> Homology(MCW,1);
[ 0, 0, 0, 0, 0, 0, 0 ]
```

Chapter 5

Further arrangement properties

The following describes functions to analyse further properties of arrangements such as *formality*, *supersolvability*, *simpliciality*, *factoredness*, *inductive factoredness* ...

5.1 Formal arrangements

5.1.1 IsFormal (for IsHyperplaneArrangement)

▷ `IsFormal(A)` (property)

Returns: true or false.

Determines whether the arrangement is *formal* in the sense of Falk-Randell [FR87].

Example

```
gap> A1 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [3,0,5],
>     [3,4,5]
> ]);;
gap>
gap> A2 := Arr([
>     [1,0,0],
>     [0,1,0],
>     [0,0,1],
>     [1,1,1],
>     [2,1,1],
>     [2,3,1],
>     [2,3,4],
>     [1,0,3],
>     [1,2,3]
> ]);;
gap> IsLEquiv(A1,A2);
true
gap> IsFormal(A1);
```

```
true
gap> IsFormal(A2);
false
```

Chapter 6

Oriented Matroids

Oriented matroids are abstractions of real hyperplane arrangements. The following describes functions to handle oriented matroids and compute associated cell complexes and invariants. See [BLVS+99].

6.1 Attributes

6.1.1 OMRank (for IsOrientedMatroid)

▷ `OMRank(OM)` (attribute)

Returns: An integer

Returns the rank of the oriented matroid *OM*.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMRank(OM);
3
```

6.1.2 OMGroundSet (for IsOrientedMatroid)

▷ `OMGroundSet(OM)` (attribute)

Returns: A list

Returns the ground set of the oriented matroid OM.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGroundSet(OM);
[ 1, 2, 3, 4, 5, 6 ]
```

6.1.3 OMLForms (for IsOrientedMatroid)

▷ `OMLForms(OM)` (attribute)

Returns: A list of linear forms or `fail`

Returns the list of linear forms associated with the oriented matroid OM. If the internal component `OM!.lforms` is not bound, `fail` is returned.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMLForms(O);
[ [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

6.1.4 OMGeomLattice (for IsOrientedMatroid)

▷ `OMGeomLattice(OM)` (attribute)

Returns: An object of type `IsGeomLattice` or `fail`

Computes the geometric lattice associated with the oriented matroid OM. If OM is linear (as determined by `OMIsLinear(OM)`), the lattice is computed as the intersection lattice of the hyperplane arrangement given by `OMLForms(OM)`, stored in the internal component `OM!.lattice`, and returned. If OM is not linear, `fail` is returned.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGeomLattice(OM);
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
[ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
  [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
```

6.1.5 OMCocircuits (for IsOrientedMatroid)

▷ `OMCocircuits(OM)` (attribute)

Returns: A list

Computes the (signed) cocircuits of the oriented matroid OM. The cocircuits are derived from the geometric lattice of OM (via `OMGeomLattice(OM)`) and the linear forms (from `OMLForms(OM)`). The result is a list of signed vectors representing all cocircuits of OM.

Example
```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]); OMCocircuits(O);
<OrientedMatroid: 3 elements, rank 2>
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
```

6.1.6 TopeGraph (for IsOrientedMatroid)

▷ `TopeGraph(OM)` (attribute)

Returns: A graph

Computes the tope graph of the oriented matroid OM. The topes are obtained as the first set of covectors from `OMCovectors(OM)`. Two topes are connected by an edge if their separating set has at most one element.

Example
```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> G := ShallowCopy(TopeGraph(O));;
gap> Vertices(G);
```

```
[ 1 .. 6 ]
gap> IsSimpleGraph(G);
true
gap> UndirectedEdges(G);
[ [ 1, 3 ], [ 1, 5 ], [ 2, 4 ], [ 2, 6 ], [ 3, 6 ], [ 4, 5 ] ]
```

6.1.7 OMCovectors (for IsOrientedMatroid)

▷ `OMCovectors(OM)` (attribute)

Returns: A list of lists

Computes all covectors of the oriented matroid OM. The result is returned as a reversed list of lists, where each inner list is a covector represented as a signed vector.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> OMCovectors(O);
[ [ [ -1, 1, -1 ], [ 1, -1, 1 ], [ -1, -1, -1 ],
    [ 1, 1, 1 ], [ -1, 1, 1 ], [ 1, -1, -1 ] ],
  [ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
    [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ],
  [ [ 0, 0, 0 ] ] ]
```

6.1.8 SalvettiComplex (for IsOrientedMatroid)

▷ `SalvettiComplex(OM)` (attribute)

Returns: A `FacePoset`

Constructs the Salvetti complex [Sal87] associated with the oriented matroid OM.

The construction proceeds as follows:

1. Computes the covectors of OM via `OMCovectors(OM)`.
2. Uses the topes and the lower order ideals of the covector poset to form the cells of the complex.
3. Defines an order function on the cells to capture the face poset structure, using `OMOperation` and `OrderCovec`.
4. Returns an object of type `FacePosetFamily` representing the Salvetti complex, with its ground set stored in `grGroundSet` and the order function in `orderfunction`.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

6.1.9 FPGroundSet (for IsFacePoset)

▷ `FPGroundSet(FP)` (attribute)

Returns: A list

Returns the ground set of the face poset FP.

Example

```
gap> S := SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

```
gap> FPGroundSet(S);
[
  [
    [ [ -1, 1, -1 ], [ -1, 1, -1 ] ],
    [ [ 1, -1, 1 ], [ 1, -1, 1 ] ],
    [ [ -1, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ 1, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ -1, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ 1, -1, -1 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ -1, 0, -1 ], [ -1, 1, -1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, -1 ] ],
    [ [ 1, 0, 1 ], [ 1, -1, 1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ -1, 0, -1 ], [ -1, -1, -1 ] ],
    [ [ 0, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ 1, 0, 1 ], [ 1, 1, 1 ] ],
    [ [ 0, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, -1, -1 ], [ 1, -1, -1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ 0, 0, 0 ], [ -1, 1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, -1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, -1 ] ]
  ]
]
```

6.1.10 FPOrder (for IsFacePoset)

▷ `FPOrder(FP)` (attribute)

Returns: A function

Returns the order function of the face poset FP.

6.1.11 OMChirotope (for IsOrientedMatroid)

▷ `OMChirotope(OM)` (attribute)

Returns: A function

Returns the chirotope giving as a function from $\binom{E}{r} \to \{0, \pm 1\}$, where E is the groundset of OM and r is the rank of `OM`.

6.1.12 OMRankFunction (for IsOrientedMatroid)

▷ `OMRankFunction(OM)` (attribute)

Returns: A function

Returns the rank function of the oriented matroid *OM*

6.2 Properties

6.2.1 OMIsLinear (for IsOrientedMatroid)

▷ `OMIsLinear(OM)` (property)

Returns: true or false

Returns whether the oriented matroid *OM* is linear. This is the case if the internal component `OM!.lforms` is bound.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMIsLinear(OM);
true
```

6.2.2 HasSimpleTriangle (for IsOrientedMatroid)

▷ `HasSimpleTriangle(OM)` (property)

Returns: true or false.

The same as `HasSimpSimplexRk(OM,3)`.

Example

6.3 Operations

6.3.1 IsLEquiv (for IsOrientedMatroid,IsOrientedMatroid)

▷ `IsLEquiv(OM1, OM2)` (operation)

Returns: true or false

Determines if the oriented matroids *OM1* and *OM2* have isomorphic geometric lattices.

Example

```
gap> O1:=OrientedMatroid(3,4,[1,1,1,1]);
<OrientedMatroid: 4 elements, rank 3>
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,1]]);
<OrientedMatroid: 4 elements, rank 3>
gap> IsLEquiv(O1,O2);
true
gap> O2:=OrientedMatroid([[1,0,0],[0,1,0],[0,0,1],[1,1,0]]);
<OrientedMatroid: 4 elements, rank 3>
gap> IsLEquiv(O1,O2);
false
```

6.3.2 HasSimpleSimplexRk (for IsOrientedMatroid, IsInt)

▷ `HasSimpleSimplexRk(OM, k)` (operation)

Returns: true or false

Returns whether the oriented matroid *OM* has a localiazion of rank k with a simple Simplex, i.e. such that the localizations at its vertices are boolean. If this is true for $k \geq 3$ then its Salvetti complex can not be $K(\pi, 1)$.

Example

6.4 Global methods

6.4.1 OMOperation

▷ OMOperation(*sv1, sv2*) (function)

Returns: A list

Performs the oriented matroid operation on two signed vectors sv1 and sv2. For each position i, if sv1[i] is nonzero, it is used; otherwise sv2[i] is used. Returns a new signed vector representing the result of the operation.

Example

```
gap> CVs:=OMCovectors(O)[2];
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
gap> sv1 := CVs[3]; sv2 := CVs[5];
[ 1, 0, 1 ]
[ -1, 1, 0 ]
gap> OMOperation(sv1, sv2);
[ 1, 1, 1 ]
gap> OMOperation(sv2, sv1);
[ -1, 1, 1 ]
```

6.4.2 OrderCovec

▷ OrderCovec(*arg*) (function)

Order function for sign vectors.

6.4.3 IsOMEquiv

▷ IsOMEquiv(*OM1, OM2*) (function)

Returns: true or false

Determines, if the oriented matroids *OM1* and *OM2* are isomorphic.

6.4.4 SeparatingSet

▷ SeparatingSet(*arg*) (function)

The separating set of two sign vectors.

6.5 Constructors

6.5.1 OrientedMatroid (for IsList)

▷ OrientedMatroid(*r, or, A, or[, n, r, ChiroCoreList]*) (operation)

Returns: An oriented matroid OM.

Constructs am oriented matroid from a list `r` of vectors representing defining linear forms of real hyperplanes or a real hyperplane arrangement `A` or a intergers `n,r` giving the size of the groundset and the rank and a list `ChiroCoreList` $\subseteq \{0,1,-1\}^m (m = \binom{n}{r})$ presenting the chirotope of the oriented matroid.

Example

```
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(Roots(A));
<OrientedMatroid: 6 elements, rank 3>
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
```

6.5.2 OrientedMatroid (for IsHyperplaneArrangement)

▷ `OrientedMatroid(arg)` (operation)

6.5.3 OrientedMatroid (for IsInt, IsInt, IsList)

▷ `OrientedMatroid(arg1, arg2, arg3)` (operation)

Chapter 7

Special arrangements

The following describes functions to construct some special arrangements.

7.1 Global methods

7.1.1 AGpql

▷ `AGpql(p, q, l)` (function)

Returns: A hyperplane arrangement.

Constructs the reflection arrangement associated to the monomial complex reflection group $G(p,q,l)$. The hyperplanes are defined by equations of the form

$$x_i = \zeta^k x_j$$

where ζ is a primitive p-th root of unity.

Example

```
gap> A := AGpql(2,1,3);
 <HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

7.1.2 SsS3

▷ `SsS3(n)` (function)

Returns: A hyperplane arrangement.

Generates the irreducible supersolvable simplicial arrangement of rank 3 with n hyperplanes. n must either be even oder congruent 1 mod 4. In any case, $n \geq 6$.

Example

```
gap> A:=SsS3(9); Roots(A);
<HyperplaneArrangement: 9 hyperplanes in 3-space>
[ [ 0, 0, 1 ], [ 0, 1, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 0 ], [ -1, 0, 0 ],
  [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 0 ], [ 1/2*E(8)-1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3,
```

```
[ -1/2*E(8)+1/2*E(8)^3, 1/2*E(8)-1/2*E(8)^3, 1 ], [ -1/2*E(8)+1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3,
[ 1/2*E(8)-1/2*E(8)^3, -1/2*E(8)+1/2*E(8)^3, 1 ] ]
```

Chapter 8

Chamber and tope posets

8.1 Operation

8.1.1 TopePoset (for IsOrientedMatroid, IsList)

▷ `TopePoset(OM, BTope)` (operation)

Returns: A TopePoset object

Constructs the tope poset of the oriented matroid *OM* with respect to the base tope *BTope* (given as sign vector).

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]);
```

8.2 Attributes

8.2.1 TPGroundSet (for IsTopePoset)

▷ `TPGroundSet(TP)` (attribute)

Returns: A list of lists.

The ground set of the tope poset *TP*. It is a list of list, recording the separating elements of the given tope from the base tope.

8.2.2 TPBaseTope (for IsTopePoset)

▷ `TPBaseTope(TP)` (attribute)

Returns: A sign vector.

Returns the bast tope of the tope poset *TP*.

References

[Abe16] T. Abe. Divisionally free arrangements of hyperplanes. *Invent. Math.*, 204(1):317–346, 2016. 12

[BLVS+99] Anders Björner, Michel Las Vergnas, Bernd Sturmfels, Neil White, and Günter M. Ziegler. *Oriented matroids*, volume 46 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, Cambridge, second edition, 1999. 3, 17

[Ell26] G. Ellis. HAP, Homological Algebra Programming, Version 1.75. `https://gap-packages.github.io/hap`, Mar 2026. GAP package. 13

[FR87] M. Falk and R. Randell. On the homotopy theory of arrangements. In *Complex analytic singularities*, volume 8 of *Adv. Stud. Pure Math.*, pages 101–124. North-Holland, Amsterdam, 1987. 15

[MY25] P. Mücksch and M. Yoshinaga. Milnor fibrations and oriented matroids. *ArXiv e-prints*, page arXiv:2508.15331, 2025. 13

[OT92] P. Orlik and H. Terao. *Arrangements of Hyperplanes*. Grundlehren der mathematischen Wissenschaften. Springer, 1992. 3

[Sal87] M. Salvetti. Topology of the complement of real hyperplanes in $\mathbf{C}^N$. *Invent. Math.*, 88(3):603–618, 1987. 5, 19

[Ter80] H. Terao. Arrangements of hyperplanes and their freeness I. *J. Fac. Sci. Univ. Tokyo*, 27:293–320, 1980. 12

Index

AGpql, 24

CharPoly
for IsHyperplaneArrangement, 5

Dimension
for IsHyperplaneArrangement, 4
DrawLatex3Arr, 8

Essentialization
for IsHyperplaneArrangement, 8
ExpArr
for IsHyperplaneArrangement, 5

FPGroundSet
for IsFacePoset, 19
FPOrder
for IsFacePoset, 20
FPtoCWCpx, 14

GLAtoms
for IsGeomLattice, 6
GLAutGroup
for IsGeomLattice, 7
GLCharPoly
for IsGeomLattice, 7
GLGraph
for IsGeomLattice, 7
GLGroundSet
for IsGeomLattice, 6
GLIsIrreducible
for IsGeomLattice, 7
GLkFlats
for IsGeomLattice, 6
GLMoebius
for IsGeomLattice, 7
GLRank
for IsGeomLattice, 6

HArrIsIrreducible
for IsHyperplaneArrangement, 6
HArrResHind, 10
HArrResX, 10
HasSimpleSimplexRk
for IsOrientedMatroid, IsInt, 21
HasSimpleTriangle
for IsOrientedMatroid, 21
HyperplaneArrangement
for IsList, 10

IntersectionLattice
for IsHyperplaneArrangement, 4
IsDivisionallyFree
for IsHyperplaneArrangement, 12
IsFormal
for IsHyperplaneArrangement, 15
IsInductivelyFree
for IsHyperplaneArrangement, 12
IsLEquiv
for IsHyperplaneArrangement,IsHyperplaneArrangement, 7
for IsOrientedMatroid,IsOrientedMatroid, 21
IsOMEquiv, 22
IsReal
for IsHyperplaneArrangement, 6

MilnorFiberComplex
for IsHyperplaneArrangement, 13
for IsOrientedMatroid, 13
MSetInvL
for IsHyperplaneArrangement, 5

OMChirotope
for IsOrientedMatroid, 20
OMCocircuits
for IsOrientedMatroid, 18
OMCovectors
for IsOrientedMatroid, 19
OMGeomLattice

for IsOrientedMatroid, 18
OMGroundSet
for IsOrientedMatroid, 17
OMIsLinear
for IsOrientedMatroid, 21
OMLForms
for IsOrientedMatroid, 17
OMOperation, 22
OMRank
for IsOrientedMatroid, 17
OMRankFunction
for IsOrientedMatroid, 20
OrderCovec, 22
OrientedMatroid
for IsHyperplaneArrangement, 23
for IsInt, IsInt, IsList, 23
for IsList, 22

Roots
for IsHyperplaneArrangement, 4

SalvettiComplex
for IsHyperplaneArrangement, 5
for IsOrientedMatroid, 19
SeparatingSet, 22
SsS3, 24

TopeGraph
for IsOrientedMatroid, 18
TopePoset
for IsOrientedMatroid, IsList, 26
TPBaseTope
for IsTopePoset, 26
TPGroundSet
for IsTopePoset, 26